Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: D. Kanishkarane
Roll No: 24011101021

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Source Code

`https://github.com/Kanishkarane/Deep-Learning/tree/main/MLP`

8. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

9. Mandatory Plots

1. Sample Images

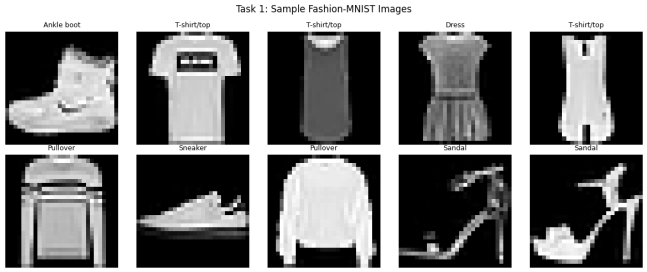


Figure 1: Sample Images from the Dataset

Inference:

The images shows the dataset has loaded properly and labelling of the dataset across the 10 classes.Even though the resolution is 28X28 and the garment shapes are differentiable,though similar images like shirt and pullover show overlap - which causes confusion.

2. Class Distribution

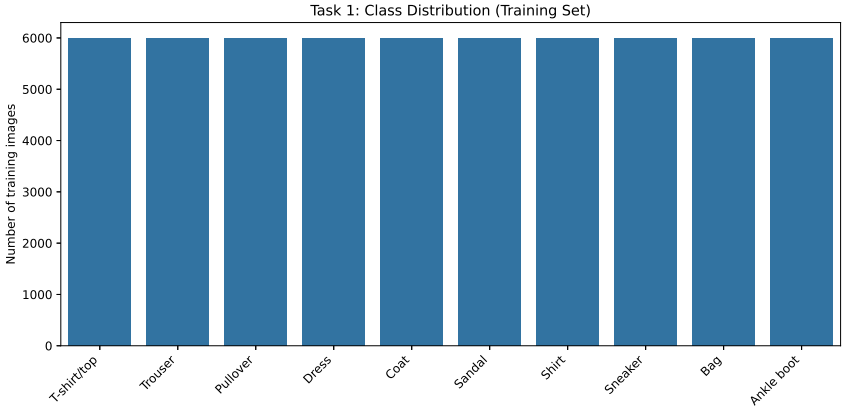


Figure 2: Class Distribution of the Dataset

Inference:

The figure shows that it is balanced with 6000 training images in the 10 classes present.This uniform distribution means that the model will not be biased towards predicting any specific class because it has more examples.There is no oversampling,undersampling or class-weighting needed here.

3. Training Accuracy vs Epoch

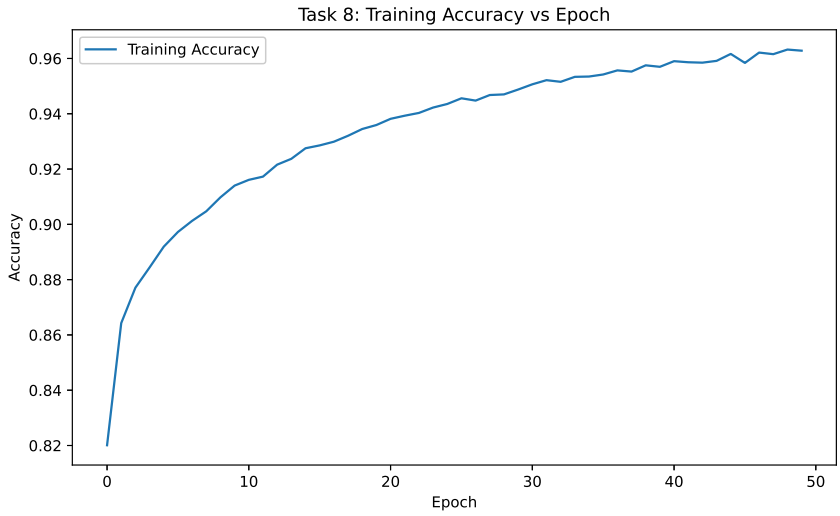


Figure 3: Training Accuracy vs Epoch

Inference:

Training accuracy rises sharply in the first 10 epochs, from around 82 to 92, then continues climbing more gradually and plateaus near 96 by the later epochs. The curve is smooth with no sudden drops, indicating stable learning, though the flattening slope after epoch 30 suggests the model is approaching its capacity on the training data and further epochs would yield only marginal gains.

4. Validation Accuracy vs Epoch

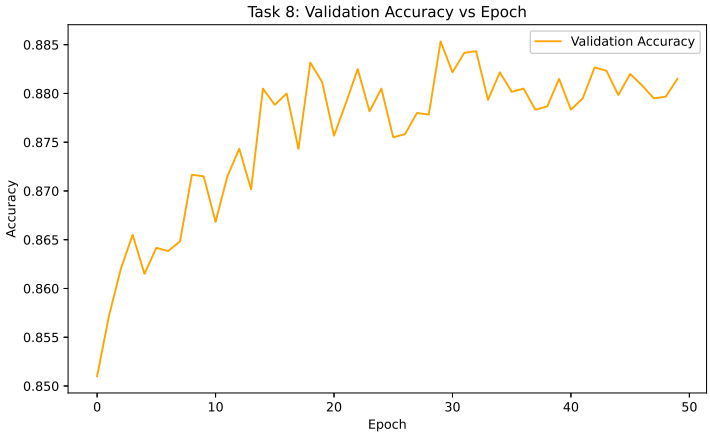


Figure 4: Validation Accuracy vs Epoch

Inference:

Training loss drops rapidly from 0.50 to 0.20 in first 10 epochs, then decreases steadily and smoothly to 0.10 at the last epoch. The constant, monotonic decrease with no plateaus means that the model is fitting the training data better and better with each epoch, consistent with the increasing training accuracy we saw earlier.

5. Training Loss vs Epoch

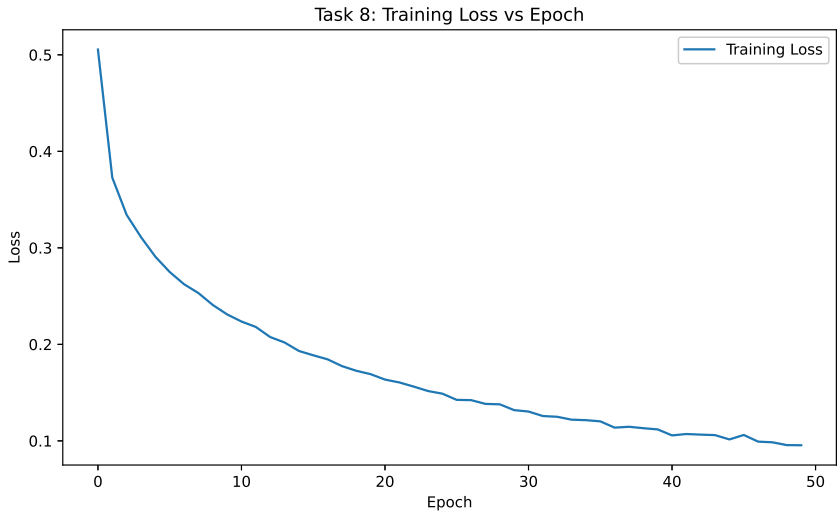


Figure 5: Training Loss vs Epoch

Inference:

Training loss rapidly falls from 0.50 to 0.20 in the first 10 epochs, then steadily and smoothly decreases to 0.10 at the last epoch. The decline is constant and monotonic (no plateaus). This is consistent with the model fitting the training data better with each epoch (as seen earlier in the rising training accuracy).

6. Validation Loss vs Epoch

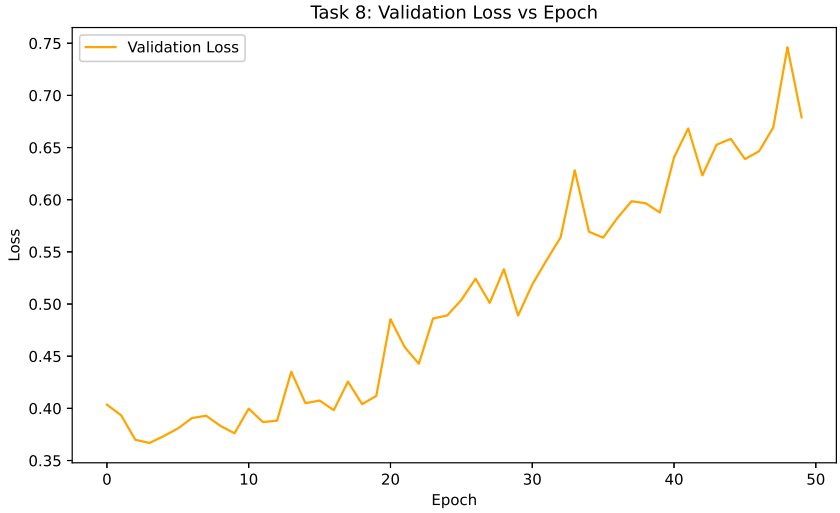


Figure 6: Validation Loss vs Epoch

Inference:

Validation loss dips a little at first but steadily starts increasing from about epoch 15 onward, going from 0.37 to over 0.70 by the final epochs. This is the most obvious overfitting signal of the entire training run – the fact that training loss continued to decline without interruption, while validation loss

moved in the opposite direction, confirms the model was memorizing training data more and more, rather than generalizing to unseen samples.

7. Confusion Matrix

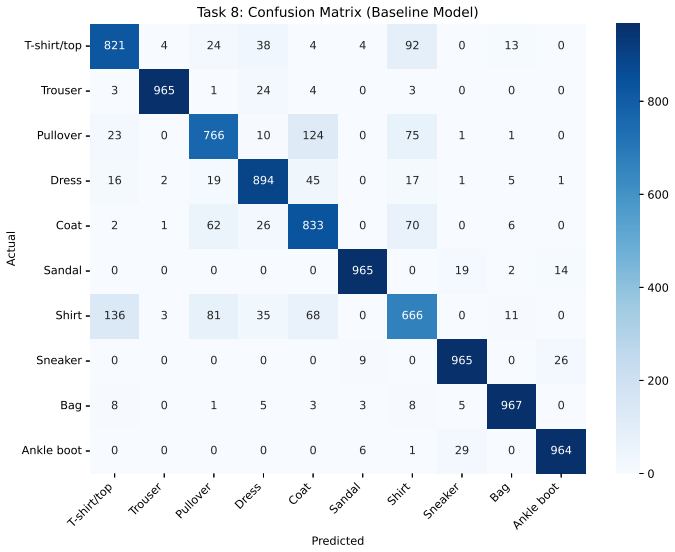


Figure 7: Confusion Matrix

Inference:

The strong diagonal indicates most classes are classified correctly, with Trouser, Sandal, Sneaker, Bag, and Ankle boot having over 96 accuracy each. As expected, the most confused is Shirt which is often misclassified as T-shirt/top (136), Pullover (81) and Coat (68) – these garments look genuinely similar in a flattened 28x28 grayscale image. Pullover and Coat also confuse with each other (124 and 62 respectively) which confirms that the main weakness of the model is not a failure of all classes, but the ability to distinguish upper-body garments with similar silhouette.

8. Hyperparameter Search Results

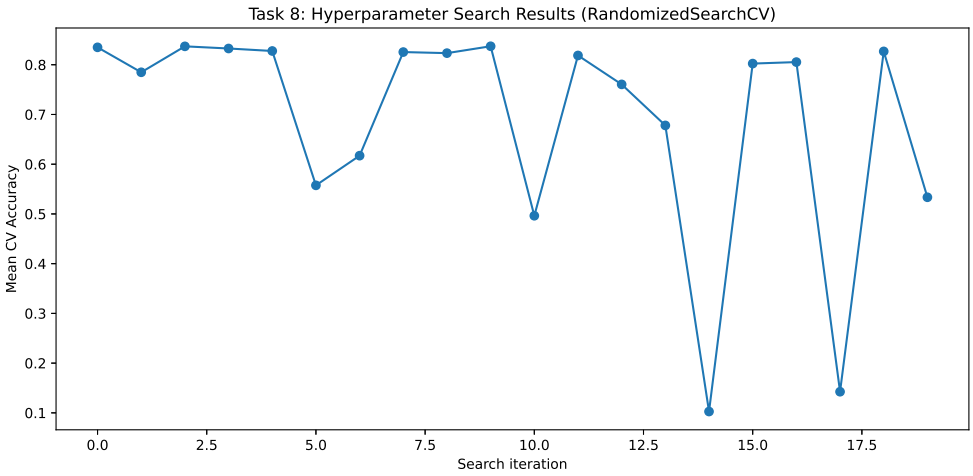


Figure 8: Hyperparameter Search Results

Inference:

The mean CV accuracy varies widely across the 20 sampled configurations, from as low as 0.10 (iteration 14) to as high as 0.84 (iterations 2, 9, 18). This large spread confirms that hyperparameter choice matters a lot for this model – combinations with certain activation functions or learning rates fail badly, while others land in a fairly tight, strong-performing cluster around 0.80-0.84, which is where the best configuration was ultimately selected from.

9. Best Model Accuracy Comparison

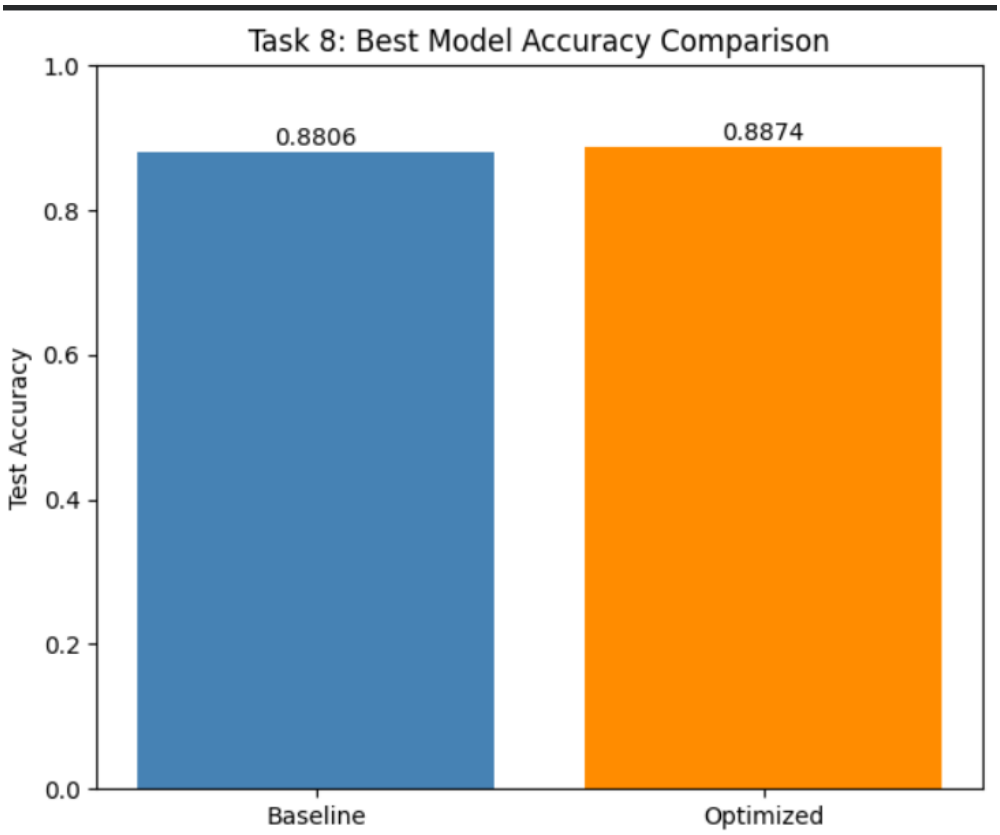


Figure 9: Best Model Accuracy Comparison

Inference:

The optimized model achieves 88.74 test accuracy against the baseline's 88.06, a small improvement of 0.68 percentage points. So this just confirms that the hyperparameter tuning gave a small, but real gain. The gap shows that the default baseline architecture in the manual was already pretty close to the optimal for this dataset, so there is not much room for further improvements by tuning alone.This confirms that hyperparameter tuning provided a small but real improvement, though the gap indicates the manual's default baseline architecture was already fairly close to optimal for this dataset.

10. Results

Best Hyperparameters

Hidden Layers	2
Hidden Neurons	256
Learning Rate	0.001
Batch Size	32
Optimizer	adam
Activation Function	sigmoid
Epochs	30
Dropout	0.0
Cross-validation Accuracy	0.8372
Testing Accuracy	0.8874

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8806	0.8874
Precision	0.8803	0.8869
Recall	0.8806	0.8874
F1-score	0.8801	0.8858
Training Time	263.96	152.32

11. Additional Tasks

1. **Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging.**
   A single-layer perceptron with a step activation function was trained on the XOR gate, using the update rule $w \leftarrow w + \eta(y - \hat{y})x$ and $b \leftarrow b + \eta(y - \hat{y})$, when a training sample was misclassified. OR, AND, and NOT gates converged easily, while for XOR the perceptron was run for 20 epochs and didn't converge, therefore producing 78 weight updates in total and never reached a pass which didnt have any errors.

Update	Input, Target	(w, b)
0 (init)	–	[0, 0], 0.00
1	(0,0), 0	[0, 0], -1.00
2	(0,1), 1	[0, 1], 0.00
3	(1,1), 0	[-1, 0], -1.00
4	(0,1), 1	[-1, 1], 0.00
5	(1,0), 1	[0, 1], 1.00
6	(1,1), 0	[-1, 0], 0.00
7	(0,0), 0	[-1, 0], -1.00
8	(0,1), 1	[-1, 1], 0.00
9	(1,0), 1	[0, 1], 1.00
10	(1,1), 0	[-1, 0], 0.00

Table 1: Weights and bias after each update for the XOR gate

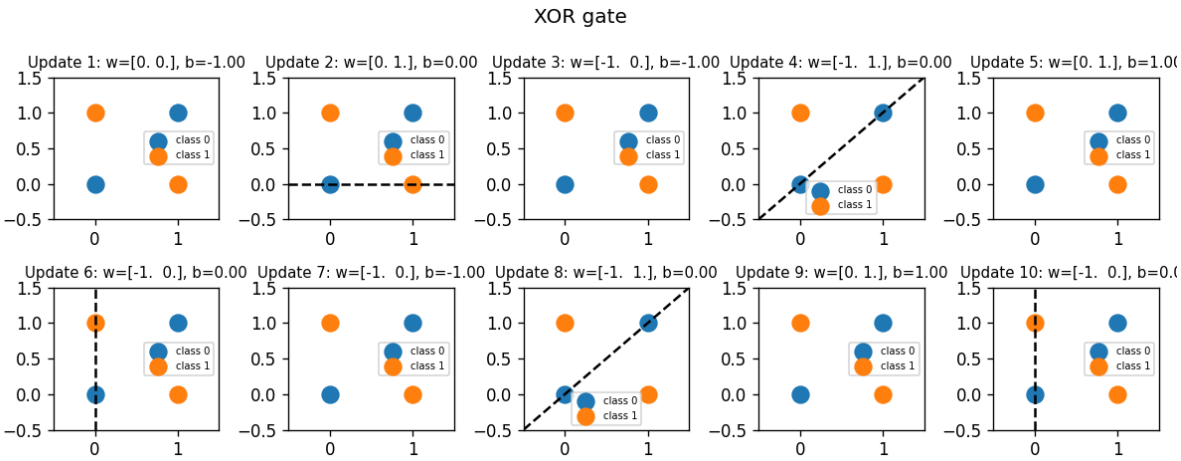


Figure 10: Decision boundary after each weight update — XOR gate (first 10 updates)

Inference / Pattern analysis: The function XOR which was implemented is not linearly separable - as points belong to class 0 and two points which belong to class 1 and it is 2 different corners of the graph.Therefore, one single line or decision boundary cannot separate it into two different classes.Thereby each weight update that gets classified correctly ends up misclassifying at least one of the other three points so the updates undoes the correction which was previously updated.THis is the reason why weight and bias are going into 4 cycle loop and doesnt converge.THe perceptron tries to draw a linear boundary which is not possible in this case.This is the drawback of perceptron which lead to the creation of multi layer perceptron(MLP) which has non linear activation functions and makes the representation of XOR possible.

12. Discussion

Answer the following:

1. Which optimization method was used?
   Randomized Search with 5-fold cross-validation (20 candidates) was used instead of Grid Search. The full grid would have needed thousands of fits, making it computationally infeasible.

2. Which hyperparameters were selected?
   2 hidden layers with 256 neurons, sigmoid activation, Adam optimizer, learning rate 0.001, batch size 32, and 30 epochs, with no dropout. This combination gave the best cross-validation score among the sampled candidates.
3. Did optimization improve performance?
   Yes, a little bit — test accuracy started rising from 88.06 (baseline) to 88.74(optimized), a gain of about 0.68 percentage points. This shows that the manual's default baseline architecture was already close to well-tuned for this dataset.
4. Which hyperparameter had the greatest impact?
   Activation function showed the most wide spread in accuracy across its values, ranging from 0.763 (tanh) to 0.143 (ReLU). Learning rate was the next most impactful, varying accuracy from 0.799 down to 0.391.
5. Compare Grid Search and Randomized Search.
   Grid Search checks each and every combination and agrees the best one within the grid, but its cost multiplies with each added hyperparameter. Randomized Search samples a fixed number of combinations instead, trading that guarantee for a search that actually finishes in reasonable time.
6. Recommend the best MLP configuration.
   The recommended configuration is 2 hidden layers of 256 neurons with sigmoid activation, trained using Adam at a learning rate of 0.001, batch size 32, for 30 epochs with no dropout. This setup achieved 88.74

13. Conclusion

A multi-layer perceptron (MLP) with the architecture of 784→128→64→10 was trained on the Fashion-MNIST dataset, and it achieved a baseline test accuracy of 88.06. By using Randomized Search along with 5-fold cross-validation, we explored various hyperparameters like network depth, width, learning rate, batch size, optimizer, activation functions, and dropout rates. This process led us to an optimized model that reached an accuracy of 88.74, which is a modest improvement of 0.68 points. This suggests that the default architecture was already quite effective for this task. Among the hyperparameters, the activation function and learning rate had the most significant impact, while batch size and dropout seemed to play a lesser role. Overall, this experiment highlights that automated hyperparameter tuning can be a valuable tool for model selection. However, when dealing with a relatively straightforward and well-balanced dataset like Fashion-MNIST, the benefits of fine-tuning an already solid baseline are somewhat limited.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.